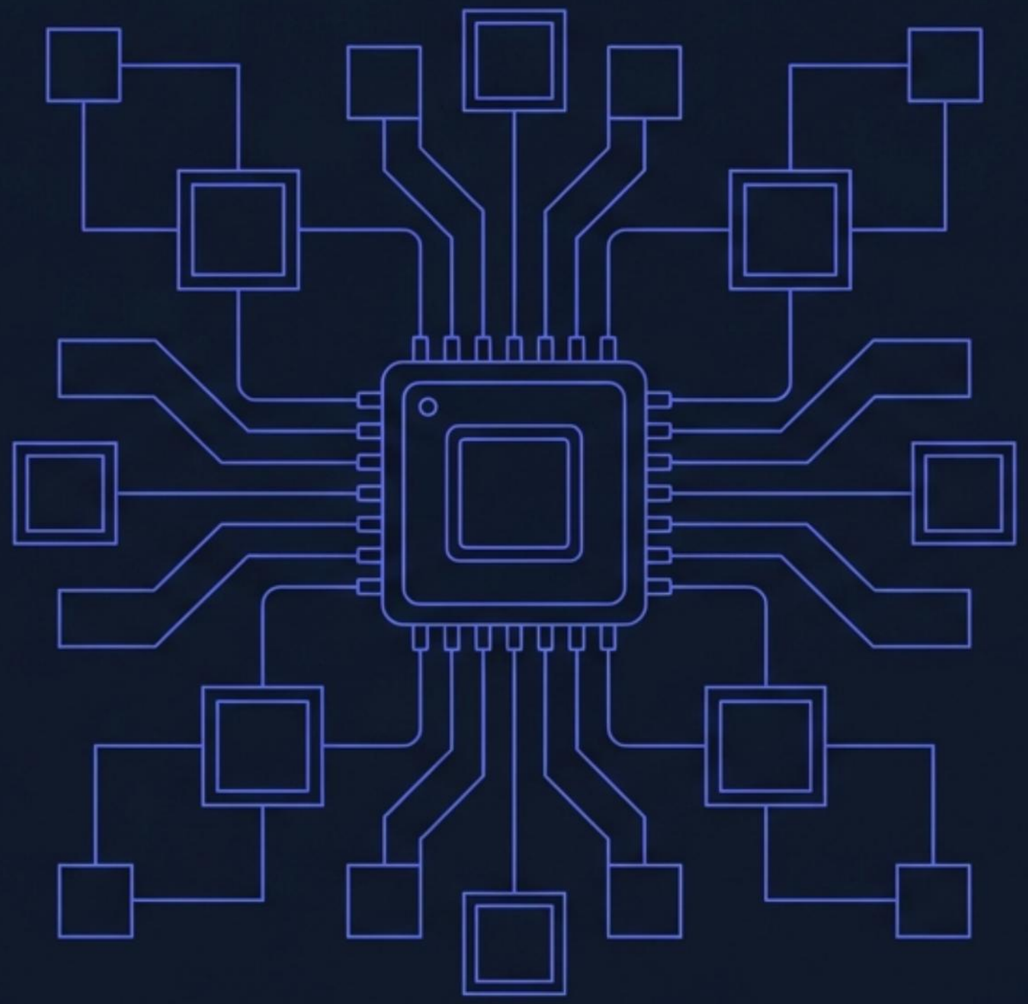


Ecosistema Digital del Club de Robótica

Arquitectura, Operación y Estrategia de Crecimiento Institucional.

Fase de Proyecto: Marzo - Abril 2026

Ingeniería en Sistemas Computacionales

Nota de Política: Operación bajo identidad independiente (Club de Robótica) en fase de pre-aprobación oficial.

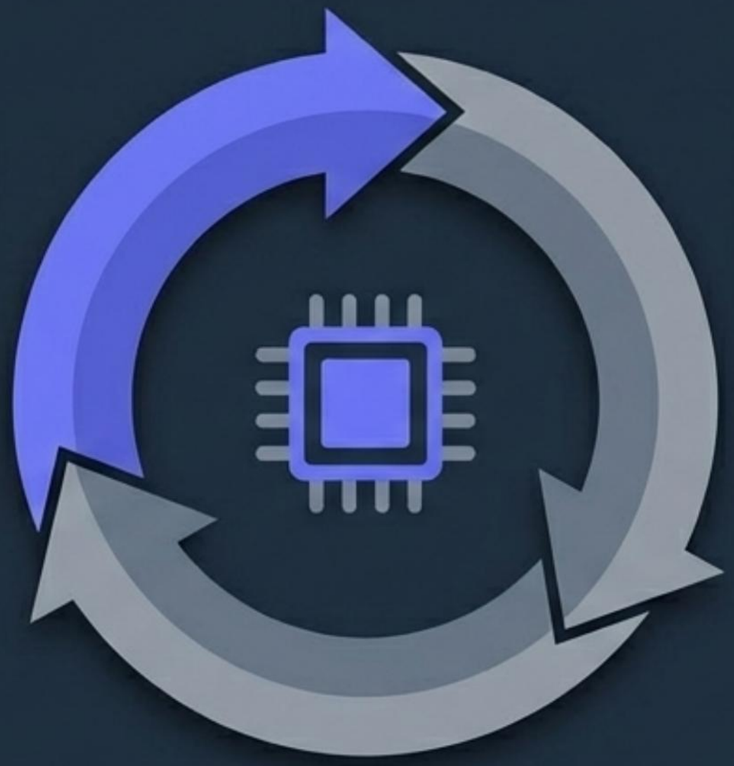
El Antes y el Después de la Gestión de Talento

El Problema		La Solución
Proyectos ocultos y nula difusión ante la comunidad.	→	Vitrina digital 24/7 para robots seguidores de línea, sumos y brazos robóticos.
Pérdida de talento por falta de canales formales.	→	Captación en laboratorio vía Códigos QR directos al portal.
Dependencia técnica para cualquier actualización.	→	Administración 100% autónoma sin necesidad de tocar código.
Dificultad para organizar equipos para certámenes.	→	Plataforma centralizada para la ruta hacia la WRO México 2026.

El Motor de Captación Continua

Fase 3: Visibilizar Logros

Publicación de resultados, galerías fotográficas y proyectos terminados. Esto crea un efecto de demostración que vuelve a alimentar la Fase 1.



Fase 1: Despertar el Interés

Actuar como un imán digital. Uso de noticias, convocatorias y un diseño UI atractivo para captar la atención inicial del alumno universitario.

Fase 2: Formalizar la Participación

Convertir el interés en acción. Registro directo de alumnos, formación de equipos e inscripción a talleres mediante un proceso sin fricción.

Hacia la Olimpiada Mundial de Robótica 2026

Puerto Rico 2026: Los robots se encuentran con la cultura. Edades 8-22 años.

Atracción Local

Códigos QR desplegados en laboratorios STEAM dirigen a los estudiantes curiosos a la plataforma web.

Consolidación de Equipos

Los usuarios registrados acceden a las reglas oficiales (RoboMission, Future Innovators) y forman equipos de trabajo en los talleres del club.

Representación Nacional

Equipos de alto rendimiento preparados para las etapas Regionales (Mayo-Agosto) y Nacionales (Septiembre) bajo la normativa de FUNDESTEAM México.

Un Ecosistema Digital Bimodal

El Frente Público (Frontend)

Para la Comunidad Universitaria.

- **Características:**
  - Diseño Mobile-First con Tailwind CSS, animaciones de entrada dinámicas (AOS.js), motor de plantillas Jinja2.
- **Módulos:**
  - Noticias, Horarios, Galería, Certámenes, Registro.

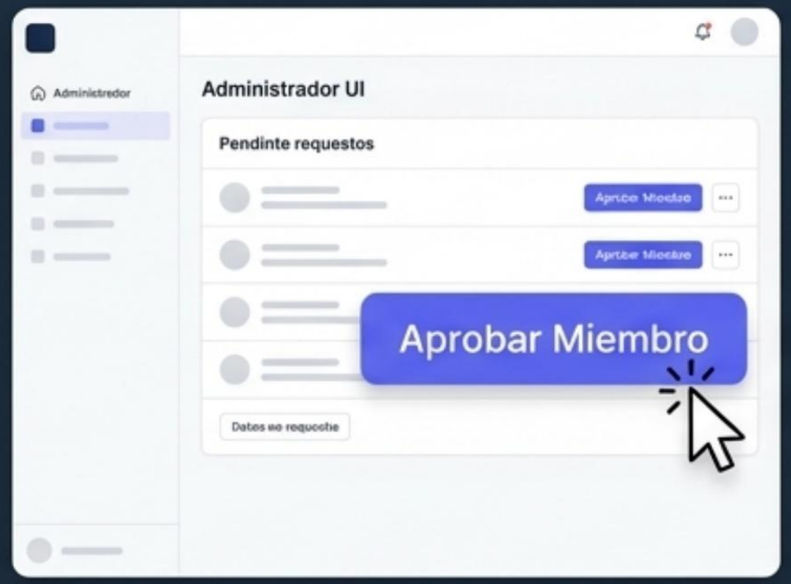
El Centro de Comando (Backend)

Para Administradores y Docentes.

- **Características:**
  - Panel seguro (Flask-Login), control de roles, operaciones CRUD completas, cero necesidad de programación.
- **Módulos:**
  - Aprobación de miembros, Publicación, Gestión WRO.

Autonomía Operativa: Flujo Zero-Code

Acción del Docente



Un solo clic en una interfaz gráfica intuitiva.

Sin intervención técnica de TI

Respuesta del Sistema

Frontend UI:
El horario se actualiza en vivo en el portal público.

Comunicaciones:
Se dispara un correo automático de bienvenida al alumno aprobado (con la clave por defecto: `robotica2026`).

Base de Datos:
El registro se consolida en MySQL de forma segura.

Motor Dinámico de Galerías y Proyectos

Flujo de Optimización

Step 1:
Entrada: Administrador sube foto cruda (Máx 5MB).

Step 2:
Proceso: La librería Pillow (Python) intercepta y comprime automáticamente.

Step 3:
Salida: Imagen optimizada en formato WebP/JPG asignada a un Álbum específico (ej. Regional 2025).

Estándares de Mantenimiento

Noticias

1200x800px. Tono educativo y entusiasta (máx 60 caracteres en títulos).

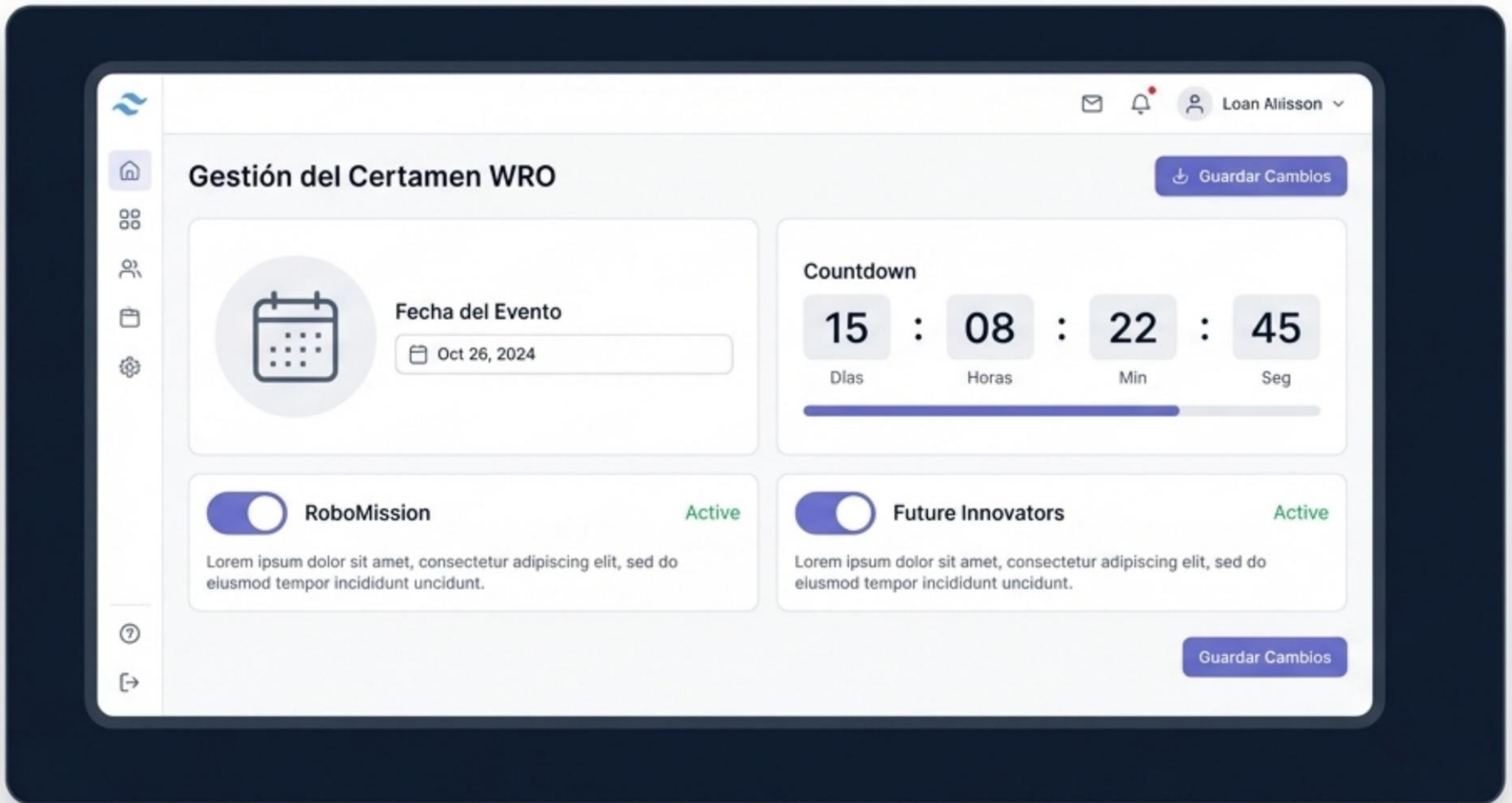
Miembros

600x600px cuadradas.

Persistencia

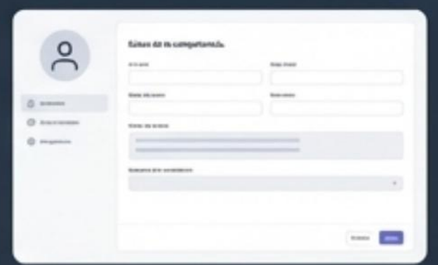
Borrado automático del servidor al eliminar el registro en la base de datos mediante listeners (`image_cleanup.py`).

Módulo Específico: Gestión del Certamen WRO 🏆



Nota de seguridad: Edición en formato seguro (JSON) aislada del código fuente principal.

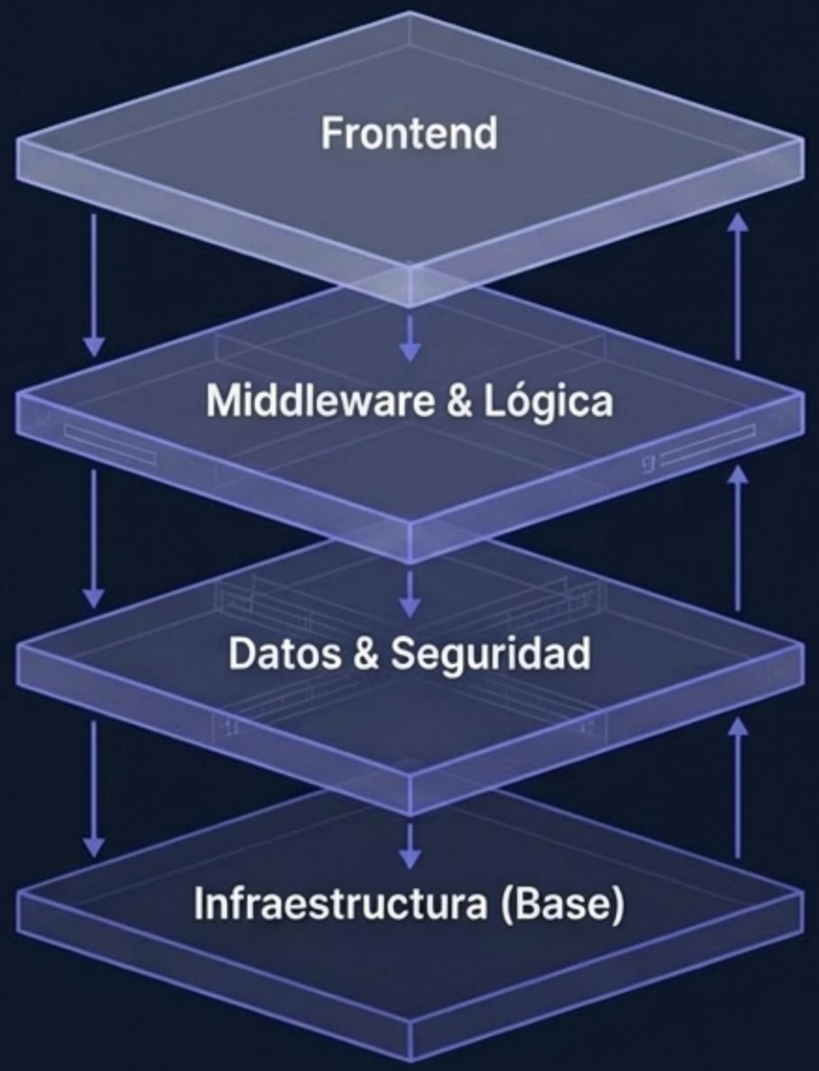
Flujo de trabajo Post-Evento



Coordinador ingresa datos de la competencia.

Actualización inmediata de resultados en la plataforma pública sin depender del departamento de sistemas.

Arquitectura de Capas (Tech Stack)



Frontend	Tailwind CSS, AOS.js, Jinja2, Material Icons. Renderizado móvil, diseño adaptativo y animaciones fluidas.
Middleware & Lógica	Python 3.12+, Flask, Flask-Admin. Enrutamiento (`main.py`), lógica de negocio y gestión de usuarios sin código.
Datos & Seguridad	SQLAlchemy (ORM), MySQL, Flask-Login, Werkzeug. Modelado de datos agnóstico, sesiones cifradas y persistencia estructurada.
Infraestructura (Base)	Hostinger / Coolify (Hetzner), Docker, Phusion Passenger. Despliegue en producción configurado mediante `Passenger_WSGI.py`.

Resiliencia, Seguridad y Persistencia de Datos

Intercepción Frontend

Flask-WTF valida y limpia los formularios contra ataques de inyección.

Procesamiento Backend

Las contraseñas se encriptan con Werkzeug (hashing); las imágenes pesadas se optimizan con Pillow.

Persistencia Aislada

Los archivos estáticos se guardan en un volumen persistente (`/app/app/static/uploads`) en Coolify, protegiéndolos de sobreescrituras en actualizaciones de código.

Respaldo Automático

Un Cron job hace un `mysqldump` regular hacia la nube, mitigando el riesgo ⚠ ALTO de pérdida de datos.

Control de versiones vía Git/GitHub (Branch `main` protegido).

Plan de Ejecución: 8 Sprints Hacia el Lanzamiento

Total de 1 mes (14 Mar - 14 Abr 2026).

Progreso Global

Bloque 1 (S1-S2): Infraestructura y Frontend

Bloque 1 (S1-S2):
Infraestructura y Frontend

- **Hito:** Entorno Flask + Páginas Base (Inicio, Horarios).

Cumplimiento de política: Sin logos UT hasta aprobación.

Bloque 2 (S3-S4): Gestión de Usuarios

Bloque 2 (S3-S4):
Gestión de Usuarios

- **Hito:** Sistema de registro y notificaciones al administrador. CRUD de noticias.

Bloque 3 (S5-S6): Contenido y Admin

Bloque 3 (S5-S6):
Contenido y Admin

- **Hito:** Galería optimizada y despliegue del panel Flask-Admin cerrado al público.

Bloque 4 (S7-S8): Despliegue y Entrega

Bloque 4 (S7-S8):
Despliegue y Entrega

- **Hito:** Sitio live con Phusion Passenger, QA, y Demostración funcional al Directorio.

Anticipación y Control de Contingencias

Incompatibilidad de Entorno (Medio)

Riesgo: Fallos al pasar de desarrollo local a producción en Hostinger.

Mitigación: Pruebas exhaustivas previas en `Passenger_WSGI.py` manteniendo el mismo stack (`Python/MySQL`).

Adopción Estudiantil Lenta (Medio)

Riesgo: La plataforma se lanza pero no capta tráfico.

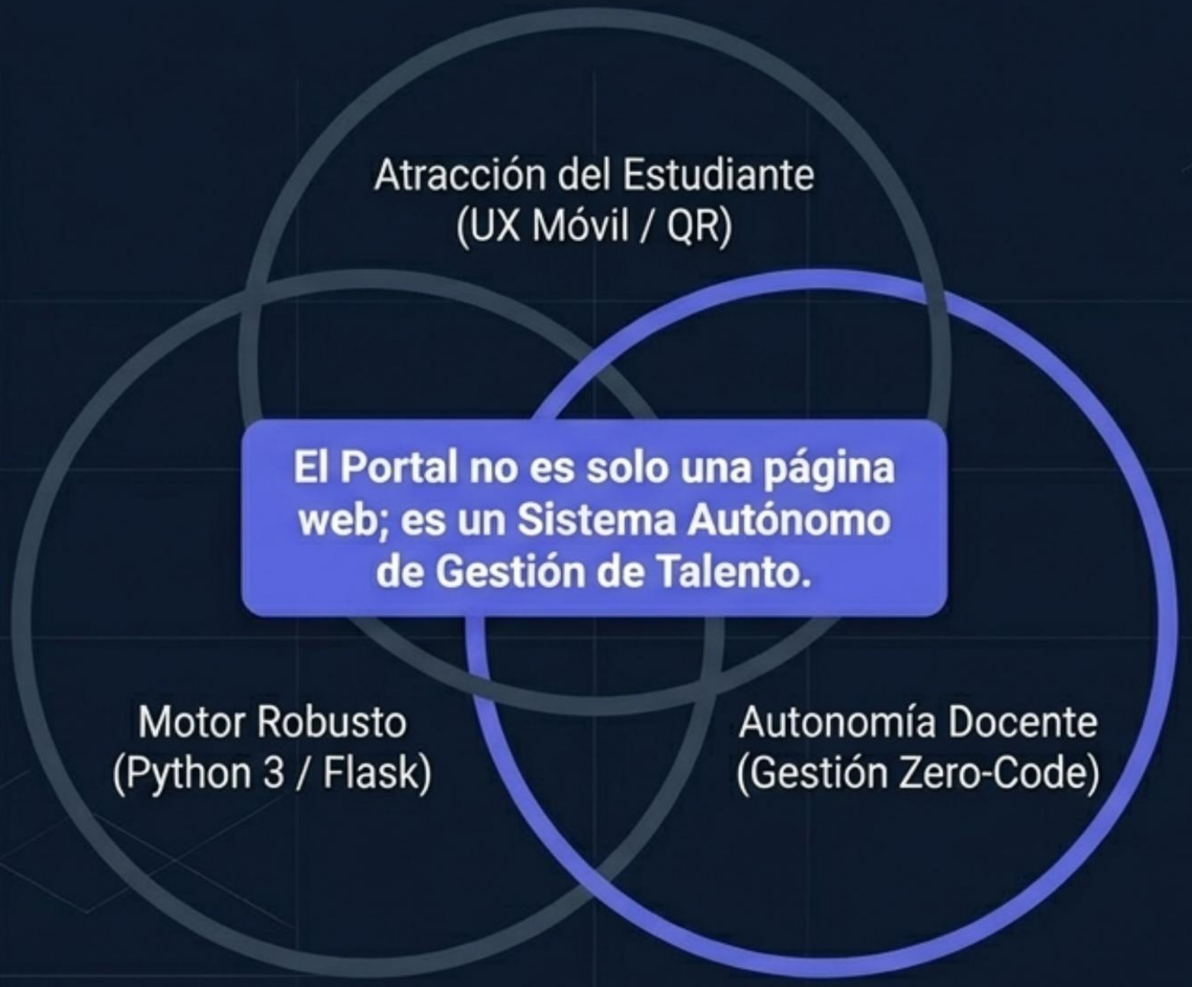
Mitigación: Despliegue de Códigos QR físicos directos en los laboratorios como gancho de inscripción inmediata.

Desviación del Alcance / Scope Creep (Medio)

Riesgo: Requerimientos extra que retrasen el plazo de 1 mes.

Mitigación: Product Backlog estrictamente priorizado y revisiones al inicio de cada uno de los 8 sprints bajo metodología ágil.

Más Allá de un Sitio Web



La convergencia de estas tres fuerzas permite a la institución escalar su presencia hacia competencias como la WRO 2026 sin generar fricción operativa ni cuellos de botella técnicos.

Definiendo y Midiendo el Éxito (Objetivo SMART)

14 Abr 2026

Fecha de Entrega

Despliegue confirmado y operativo tras 8 sprints ágiles.

≥ 50 Alumnos

Tracción Inicial

Registrados orgánicamente durante el primer mes de operación.

< 2 Segundos

Rendimiento Técnico

Tiempo de carga auditado mediante Google Lighthouse / PageSpeed.

≥ 70%

Estabilidad del Sistema

Cobertura de pruebas en la matriz QA para módulos críticos.

Aprobación Institucional / Demo Final

Sistema Listo para Despliegue

Especificaciones de Core

- **Motor:** Python 3.12+ / `Flask 3.x`
- **Arquitectura:** Bimodal (UI Pública + Admin Segura)
- **Despliegue:** Listo para entornos Docker/Nixpacks (`Coolify`).

Directrices de UI/UX

Identidad Visual: Sistema Premium Dark.

Compromiso Institucional: Transición sin fricción operativa.